

Resilient and Cost-Efficient Job Scheduling for Heterogeneous Data Centers: A Soft-Constrained Approach

Hengwen Xu
Business School
Central South University
Changsha, China
hengwenxu@csu.edu.cn

Hao Yin*
The Research Center for Digital Production
and Smart Logistics, Tsinghua University
Beijing, China
yinh24@mails.tsinghua.edu.cn

Abstract—Conventional rigid scheduling models often struggle to maintain feasibility under peak loads or resource scarcity, largely due to inflexible physical and grid constraints. To overcome this, we propose a resilient scheduling framework for heterogeneous data centers that recasts these hard boundaries within a soft-constrained Mixed-Integer Linear Programming (MILP) formulation. We tackle the computational intractability of this NP-hard problem by introducing three novel classes of valid cuts: startup validity, rapid cycle prevention, and min-down lookahead, which effectively tighten the linear relaxation. Experiments show that these cuts reduce solution times for large-scale instances by over 60% compared to standard baselines. Furthermore, sensitivity analysis confirms the model’s resilience, ensuring operational continuity and graceful degradation even when power availability drops to 50% of peak demand.

Index Terms—Heterogeneous Data Centers, Resilient Scheduling, Soft-Constrained Optimization, Mixed-Integer Linear Programming (MILP), Valid Cuts, Graceful Degradation.

I. INTRODUCTION AND RELATED WORK

Balancing operational efficiency with physical infrastructure limitations is a fundamental challenge in modern data centers. As data centers account for a growing share of global electricity demand [1], operators face increasing pressure to reduce costs. However, approaches that treat infrastructure purely as an abstraction often fail to account for the mechanical wear and electrical degradation caused by dynamic operations [2].

Energy-efficient scheduling has been widely studied. Foundational works on dynamic consolidation [3] and electricity price arbitrage [4], [5] demonstrate significant cost savings. However, these idealized models frequently overlook the penalties associated with state transitions. Lin *et al.* [6] highlight that server cycling incurs substantial costs—reflecting both energy lag and hardware wear—which can negate the savings from dynamic power management.

Grid interaction adds another layer of complexity. To ensure stability, utilities impose strict power ramping limits, which are typically modeled as rigid “hard” constraints in demand response literature [7]–[9]. In production environments prone to traffic bursts, such rigid boundaries often render the scheduling problem mathematically infeasible, causing solver failures precisely when availability is most critical.

A robust scheduler must prioritize feasibility via controlled degradation rather than system failure. While soft constraints are standard in real-time operating systems [10], their adoption in holistic data center scheduling remains limited [11], [12]. To bridge this gap, we propose a resilient scheduling framework anchored in a penalty-based optimization model. By reformulating constraints on SLAs, power capacity, ramping rates, and carbon emissions as soft constraints, our model maintains feasibility even under extreme loads. Furthermore, to address the computational complexity of this formulation, we derive valid cuts that tighten the search space and enhance solvability.

II. FORMULATION

A. Problem Description

We tackle joint scheduling and resource management in heterogeneous data centers subject to strict physical and grid constraints. The core challenge involves assigning batch jobs to servers with diverse power and wear characteristics while adhering to dynamic ramping and carbon limits. To ensure solvability during peak loads, we relax hard boundaries on capacity, deadlines, and load balancing into soft constraints. Our objective is to minimize the generalized total cost by balancing operational expenditures, including electricity and hardware wear, against weighted penalties for constraint violations, thereby ensuring system resilience.

B. Mathematical Model

Table I summarizes the sets, parameters, and decision variables used in the model. The Mixed-Integer Linear Programming (MILP) formulation for the resilient scheduling problem is presented below:

$$\begin{aligned} \min Z = & \sum_{t \in T} \sum_{k \in S} (c_t^{\text{elec}} P_{k,t}^{\text{total}} + \beta_k c^{\text{wear}} u_{k,t} + c^{\text{start}} z_{k,t}) \\ & + \sum_{j \in J} \rho^{\text{SLA}} v_j^{\text{SLA}} + \sum_{t \in T} \rho^{\text{pow}} v_t^{\text{pow}} \\ & + \sum_{t \in T} \rho^{\text{ramp}} v_t^{\text{ramp}} + \rho^{\text{carb}} v^{\text{carb}} \end{aligned}$$

TABLE I
NOMENCLATURE AND PARAMETER DEFINITIONS

Symbol	Definition
Sets and Indices	
S, k	Set of heterogeneous servers, index $k \in \{1, \dots, n\}$
J, j, i	Set of batch jobs, indices $j, i \in \{1, \dots, m\}$
T, t	Set of discrete time slots, index $t \in \{1, \dots, T \}$
Job & Workload Parameters	
$D_{j,k}$	Processing time of Job j on Server k
DL_j	Soft deadline for Job j
System & Physical Parameters	
P_k^{idle}	Power consumption of Server k (Idle/ON)
P_k^{peak}	Power consumption of Server k (Full Load)
e_t	Carbon emission intensity at time t
$C_{\text{DC}}^{\text{max}}$	Maximum aggregate power capacity
$C_{\text{carb}}^{\text{max}}$	Maximum allowable carbon emission cap
ΔP_{ramp}	Maximum allowable power ramping rate
$T_{\text{min}}^{\text{on}}$	Minimum continuous ON time
$T_{\text{min}}^{\text{off}}$	Minimum continuous OFF time
Δ^{bal}	Target threshold for load imbalance
Cost & Penalty Coefficients	
c_t^{elec}	Time-varying electricity price at time t
c^{wear}	Base hardware wear cost coefficient
c^{start}	Single server startup cost
β_k	Wear sensitivity weight for Server k
ρ^{SLA}	Penalty per unit of deadline violation
ρ^{pow}	Penalty per unit of power capacity violation
ρ^{ramp}	Penalty per unit of ramping rate violation
ρ^{carb}	Penalty per unit of carbon cap violation
ρ^{hw}	Penalty for min ON/OFF violation
ρ^{bal}	Penalty per unit of load imbalance violation
Decision Variables	
$st_{j,k,t}$	Binary: 1 if Job j starts on k at t
$x_{j,k,t}$	Binary: 1 if Job j executes on k at t
$u_{k,t}$	Binary: 1 if Server k is ON at t
$z_{k,t}$	Binary: 1 if Server k starts up at t
$P_{k,t}^{\text{total}}$	Total power of Server k at t
$v^{\dots}$	Slack variables for violations (e.g., $v_{k,k'}^{\text{bal}}$)

$$\begin{aligned}
& + \sum_{t \in T} \sum_{k \in S} \rho^{\text{hw}} (v_{k,t}^{\text{on}} + v_{k,t}^{\text{off}}) \\
& + \sum_{k \in S} \sum_{k' > k} \rho^{\text{bal}} v_{k,k'}^{\text{bal}} \\
\text{s.t. } & P_{k,t}^{\text{total}} = P_k^{\text{idle}} u_{k,t} + (P_k^{\text{peak}} - P_k^{\text{idle}}) \sum_{j \in J} x_{j,k,t}, \\
& \forall k, t \\
& \sum_{k \in S} \sum_{t=1}^{|T|} st_{j,k,t} = 1, \quad \forall j \in J \\
& x_{j,k,t} = \sum_{\tau=\max(1, t-D_{j,k}+1)}^t st_{j,k,\tau}, \quad \forall j, k, t \\
& \sum_{j \in J} x_{j,k,t} \leq 1, \quad \forall k \in S, t \in T \\
& x_{j,k,t} \leq u_{k,t}, \quad \forall j, k, t \\
& u_{k,t} - u_{k,t-1} \leq z_{k,t}, \quad \forall k, t \geq 2
\end{aligned} \tag{1}$$

$$\sum_{k \in S} \sum_{t \in T} (t + D_{j,k} - 1) st_{j,k,t} \leq DL_j + v_j^{\text{SLA}}, \quad \forall j \tag{8}$$

$$\sum_{k \in S} P_{k,t}^{\text{total}} \leq C_{\text{DC}}^{\text{max}} + v_t^{\text{pow}}, \quad \forall t \in T \tag{9}$$

$$\sum_{k \in S} (P_{k,t}^{\text{total}} - P_{k,t-1}^{\text{total}}) \leq \Delta P_{\text{ramp}} + v_t^{\text{ramp}}, \quad \forall t \geq 2 \tag{10}$$

$$\sum_{k \in S} (P_{k,t-1}^{\text{total}} - P_{k,t}^{\text{total}}) \leq \Delta P_{\text{ramp}} + v_t^{\text{ramp}}, \quad \forall t \geq 2 \tag{11}$$

$$\sum_{\tau=t-T_{\text{min}}^{\text{on}}+1}^t z_{k,\tau} \leq u_{k,t} + v_{k,t}^{\text{on}}, \quad \forall k, t \tag{12}$$

$$\begin{aligned}
& u_{k,t-1} - u_{k,t} \leq 1 - u_{k,\tau} + v_{k,t}^{\text{off}}, \\
& \forall k, \forall \tau \in \{t+1, \dots, \min(t+T_{\text{min}}^{\text{off}}, |T|)\}
\end{aligned} \tag{13}$$

$$\sum_{t \in T} \sum_{k \in S} P_{k,t}^{\text{total}} \cdot e_t \leq C_{\text{carb}}^{\text{max}} + v^{\text{carb}} \tag{14}$$

$$\sum_{t \in T} u_{k,t} - \sum_{t \in T} u_{k',t} \leq \Delta^{\text{bal}} + v_{k,k'}^{\text{bal}}, \quad \forall k, k' \in S \tag{15}$$

$$\sum_{t \in T} u_{k',t} - \sum_{t \in T} u_{k,t} \leq \Delta^{\text{bal}} + v_{k,k'}^{\text{bal}}, \quad \forall k, k' \in S \tag{16}$$

$$st, x, u, z \in \{0, 1\}; \quad v^{\dots} \geq 0 \tag{17}$$

The objective function (1) minimizes the Generalized Total Cost, which comprises electricity expenses, hardware wear, startup penalties, and weighted penalties for soft constraint violations. Regarding physical operations, Constraints (2)–(6) define power consumption, ensure unique job assignments, and restrict execution to active servers, while Eq. (7) governs the startup logic.

We formulate the soft constraints in (8)–(16). Eq. (8) permits deadline violations subject to a penalty. To ensure grid stability and hardware longevity, Constraints (9)–(13) regulate power capacity, ramping rates, and minimum ON/OFF durations. Finally, (14) and (15)–(16) address environmental and load balancing targets, allowing controlled deviations to preserve feasibility.

III. ALGORITHM DESIGN

A. Branch-and-Cut Algorithm Framework

We solve the NP-hard MILP model formulated in Section II using a Branch-and-Cut (B&C) algorithm [13]. The integrality constraints on binary variables (e.g., $x_{j,k,t}$, $z_{k,t}$) are relaxed to continuous intervals $[0, 1]$ following standard integer programming principles [14]. We employ a *Best-Bound Search* strategy for node selection to prioritize promising regions.

Within this framework, a separation procedure dynamically generates valid cuts violated by the current fractional solution x^* to tighten the relaxation. If no violated cuts are found, we apply a *most-fractional branching strategy*, selecting the

Algorithm 1 Branch-and-Cut Algorithm Framework**Require:** MILP Model parameters (Jobs J , Servers S , Time T)**Ensure:** Optimal Schedule S^* and Objective Value Z^*

```

1: Initialize: Global Upper Bound  $UB \leftarrow \infty$ , Lower Bound  $LB \leftarrow -\infty$ 
2: Create root node  $N_0$  with LP relaxation;  $\mathcal{L} \leftarrow \{N_0\}$ 
3: while  $\mathcal{L} \neq \emptyset$  do
4:   Select node  $N_k \in \mathcal{L}$  using Best-Bound strategy;  $\mathcal{L} \leftarrow \mathcal{L} \setminus \{N_k\}$ 
5:   repeat
6:     Solve LP relaxation of  $N_k$  to obtain  $x_k^*$  and value  $Z_{LP}$ 
7:     if LP is infeasible or  $Z_{LP} \geq UB$  then
8:       Prune node  $N_k$ ; break
9:     end if
10:    if  $x_k^*$  is integral then
11:      if  $Z_{LP} < UB$  then
12:         $UB \leftarrow Z_{LP}$ ;  $S^* \leftarrow x_k^*$ 
13:      end if
14:      Prune by optimality; break
15:    end if
16:    Separation Procedure: Check violations of Eqs. (18), (19), (20)
17:    if Violated cuts found then
18:      Add cuts to  $N_k$ ; continue (re-solve LP)
19:    else
20:      break (proceed to branching)
21:    end if
22:  until Node is pruned or ready for branching
23:  if  $N_k$  is not pruned then
24:    Select fractional variable  $v$  (priority to  $x_{j,k,t}$ )
25:    Generate children  $N_k^0(v=0)$ ,  $N_k^1(v=1)$ ; Add to  $\mathcal{L}$ 
26:  end if
27: end while
28: return  $S^*, UB$ 

```

variable with the highest integer infeasibility to partition the search space.

B. Valid Cuts

To accelerate convergence, we introduce three classes of valid cuts that explicitly forbid physically or logically invalid state transitions often found in fractional LP solutions.

$$z_{k,t} \leq u_{k,t}, \quad \forall k \in S, t \in T \quad (18)$$

$$z_{k,t} + z_{k,t+1} \leq 1, \quad \forall k \in S, t \in \{1, \dots, |T| - 1\} \quad (19)$$

$$u_{k,t-1} - u_{k,t} + u_{k,t+1} \leq 1, \quad \forall k, t \in \{2, \dots, |T| - 1\} \quad (20)$$

We strengthen the LP relaxation by introducing three classes of valid cuts. The Startup Validity Cut in Eq. 18 enforces logical consistency by linking startup costs to operational states. This constraint ensures that a startup cost $z_{k,t}$ is not assigned to an inactive server where $u_{k,t} = 0$. Next, the

Rapid Cycle Prevention Cut in Eq. 19 addresses numerical anomalies by eliminating fractional “micro-cycling”. It explicitly prohibits startup initiation in consecutive time slots, a condition often overlooked in standard relaxations. Lastly, the Min-Down Lookahead Cut in Eq. 20 prevents hazardous oscillation patterns such as rapid ON-OFF-ON sequences. This inequality tightens the minimum downtime constraint within the separation loop; specifically, if a server transitions from ON at $t - 1$ to OFF at t , it is forced to remain OFF at $t + 1$, effectively imposing a minimum downtime of 2 slots. Algorithm 1 details the full procedure.

IV. NUMERICAL EXPERIMENTS

A. Experimental Setup and Instances

We conducted experiments on a workstation with an AMD Ryzen 7 5800H CPU (3.20 GHz) and 32 GB RAM. We implemented all algorithms in Python 3.10.5 and solved the MILP models using Gurobi Optimizer 11.0.1 with a MIP gap tolerance of 0.01%. To validate performance across varying complexities, we generated a dataset of 15 instances categorized into small, medium, and large scales. The workload reflects real-world heterogeneity, where servers possess distinct idle power P_k^{idle} and peak power P_k^{peak} characteristics. Additionally, we introduced random noise into job processing times $D_{j,k}$ to simulate architectural affinity. Dynamic environmental factors, such as electricity prices c_t^{elec} and carbon intensity e_t , incorporate realistic peak-valley fluctuations to test the temporal scheduling capabilities of the model.

TABLE 2
SCALE DEFINITIONS OF TEST INSTANCES

Scale	Servers ($ S $)	Jobs ($ J $)	Time Horizon ($ T $)	Count
Small	3	15	10	5
Medium	5	24	12	5
Large	10	40	20	5

B. Parameter Settings and Comparative Schemes

The generalized total cost combines operational expenditures and penalties for soft constraint violations. We calibrated the penalty coefficients ρ to significantly exceed standard operational costs, ensuring that the solver prioritizes physical feasibility and only relaxes constraints under extreme resource contention. To simulate resource scarcity, we capped the power capacity $C_{\text{DC}}^{\text{max}}$ at 70% of peak demand. We benchmark two solver configurations: the *Base* model, representing the standard MILP formulation, and the *Base+Cuts* model, which incorporates the valid cuts defined in Eq. 19, 18, and 20 to tighten the linear relaxation. The solver time limits are set to 20, 100, and 600 seconds for Small, Medium, and Large instances, respectively.

C. Experimental Results and Analysis

Table 3 details the computational performance of both the standard formulation and the proposed cut-augmented model,

focusing on solution time and the number of explored Branch-and-Cut nodes. A fundamental observation across all fifteen instances is that both configurations successfully converged to the globally optimal solution with a MIP gap of 0.00%. This consistency confirms that the introduction of valid cuts strictly tightens the linear relaxation space without pruning any feasible integer solutions. Consequently, the primary metrics for evaluation are the reduction in search space (Nodes) and the resulting algorithmic efficiency (Time).

TABLE 3
DETAILED EXPERIMENTAL RESULTS OF ALL CONFIGURATIONS ON TEST INSTANCES

Scale	ID	Method	Nodes	Time (s)
Small	1	Base	27,433	3.3
		Base+Cuts	10,741	1.8
	2	Base	51,487	6.2
		Base+Cuts	19,640	3.2
	3	Base	8,424	2.3
		Base+Cuts	6,198	1.5
	4	Base	96,131	5.8
		Base+Cuts	32,191	2.1
	5	Base	7,541	1.3
		Base+Cuts	756	0.4
Medium	1	Base	49,362	30.5
		Base+Cuts	17,030	13.0
	2	Base	107,799	41.4
		Base+Cuts	26,346	16.3
	3	Base	8,587	11.8
		Base+Cuts	4,352	4.6
	4	Base	7,614	9.9
		Base+Cuts	6,757	5.5
	5	Base	17,757	16.0
		Base+Cuts	10,539	10.1
Large	1	Base	20,526	509.1
		Base+Cuts	17,921	313.1
	2	Base	11,345	482.2
		Base+Cuts	4,279	182.3
	3	Base	55,165	451.6
		Base+Cuts	83,898	387.8
	4	Base	111,872	403.2
		Base+Cuts	65,673	222.9
	5	Base	6,635	524.9
		Base+Cuts	8,599	515.2

Note: All instances reach the optimal solution (OPT).

The analysis of node counts reveals the mechanism behind the performance improvements. In small and medium-scale instances, the Base+Cuts configuration significantly reduced the size of the search tree. For example, in the medium-scale dataset, the average number of explored nodes decreased by approximately 48%. This reduction indicates that the valid cuts effectively pruned the fractional solutions that violated physical logic (e.g., rapid cycling or invalid startups), thereby allowing the solver to discard suboptimal branches earlier in the search process. This pruning capability directly translates to the observed runtime improvements, where the Base+Cuts configuration achieved an average speedup of 42.7% over the standard model for medium instances.

In large-scale experiments, which represent the core test scenario for practical operations, the scalability benefits are further pronounced. While the standard Base model approached the 600-second time limit in multiple cases, the Base+Cuts configuration demonstrated substantial resilience. Notably, in Instance 2, the node count dropped by over 62% (from 11,345 to 4,279), facilitating a corresponding reduction in solution time from 482.2s to 182.3s. Even in instances where the node count reduction was less dramatic or slightly inverted due to heuristic variations (e.g., Instance 3), the solver still maintained a lower runtime, suggesting that the cuts improved the quality of the lower bounds and guided the search towards the optimal solution more efficiently. These results validate the synergistic effect of the proposed cuts: by explicitly forbidding illogical state transitions within the LP relaxation, the framework achieves a critical balance, minimizing the generalized total cost while ensuring operational feasibility with significantly reduced computational overhead.

V. SENSITIVITY ANALYSIS

To evaluate the scalability and robustness of the proposed Base+Cuts configuration, we conducted a comprehensive sensitivity analysis varying the problem scale and resource constraints. The experiments used a baseline setting of $|S| = 5$, $|J| = 24$, and $|T| = 12$ with a capacity ratio $\eta = 0.7$, averaging results over three independent runs with a 300-second timeout. The results, visualized in Fig. 1, reveal divergent behaviors across different dimensions.

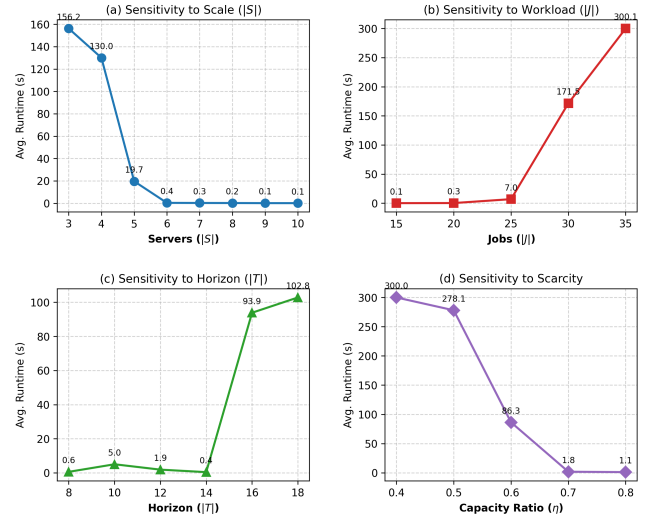


Fig. 1. Sensitivity analysis of solver runtime across four dimensions: (a) number of servers $|S|$, (b) number of jobs $|J|$, (c) time horizon $|T|$, and (d) capacity ratio η . The solver time limit is set to 300s.

The interaction between resource availability and computational complexity presents a notable inverse correlation in the server dimension. As shown in Fig. 1(a), restricted infrastructure ($|S| = 3$) forces the solver to navigate a highly constrained search space, resulting in runtimes exceeding 150 seconds. However, increasing the server count relaxes these

feasibility boundaries, allowing the Branch-and-Cut algorithm to identify low-cost schedules almost instantaneously ($< 0.5s$) when $|S| \geq 6$. Conversely, the job dimension (Fig. 1(b)) dictates the primary computational bottleneck. The system exhibits stable performance for workloads up to 25 jobs, beyond which the combinatorial explosion of assignment variables causes runtimes to spike, consistently hitting the time limit at $|J| = 35$. The time horizon (Fig. 1(c)) shows a more moderate impact, with complexity growing manageably until longer planning periods ($|T| \geq 16$) introduce deeper search trees.

Crucially, the capacity ratio analysis (Fig. 1(d)) highlights the model's resilience under stress. In scenarios of extreme scarcity ($\eta \leq 0.5$), where traditional rigid models would likely return infeasible statuses, our soft-constrained framework continues to optimize penalty trade-offs throughout the 300-second window. This high computational cost reflects the difficulty of minimizing violations in a saturated system. As resources normalize ($\eta \geq 0.7$), the pressure on soft constraints alleviates, returning the solver to high efficiency. This confirms that the proposed approach successfully achieves graceful degradation under critical load conditions while maintaining speed during normal operations.

VI. CONCLUSION

This study addressed the feasibility challenges inherent in conventional rigid scheduling models by proposing a soft-constrained MILP framework for heterogeneous data centers. By reformulating strict physical and grid limitations into penalty-based objectives, the proposed model ensures operational continuity even under peak load conditions where traditional methods typically fail.

To overcome the computational complexity associated with NP-hard scheduling problems, we derived and integrated three classes of valid cuts: startup validity, rapid cycle prevention, and min-down lookahead constraints. Numerical experiments demonstrate that these structural enhancements significantly tighten the linear relaxation, yielding a computational speedup of over 60% in large-scale instances while maintaining global optimality.

Furthermore, sensitivity analysis reveals the distinct resilience of this approach. While computational costs inevitably rise with workload density, the system successfully achieves graceful degradation under extreme resource scarcity. Instead of returning infeasible statuses during power capacity shortages, the scheduler optimizes the trade-off between service levels and operational penalties. This capability confirms that the proposed framework offers a practical, scalable solution for modern data centers facing increasing uncertainty in energy supply and workload volatility.

REFERENCES

- [1] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, "Recalibrating global data center energy-use estimates," *Science*, vol. 367, no. 6481, pp. 984–986, 2020.
- [2] R. Buyya, A. Beloglazov, and J. Abawajy, "Energy-efficient management of data center resources for cloud computing: a vision, architectural elements, and open challenges," *arXiv preprint arXiv:1006.0308*, 2010.
- [3] A. Beloglazov, J. Abawajy, and R. Buyya, "Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing," *Future generation computer systems*, vol. 28, no. 5, pp. 755–768, 2012.
- [4] Y. Zhang, Y. Wang, and X. Wang, "Greenware: Greening cloud-scale data centers to maximize the use of renewable energy," in *ACM/IFIP/USENIX international conference on distributed systems platforms and open distributed processing*. Springer, 2011, pp. 143–164.
- [5] L. Rao, X. Liu, L. Xie, and W. Liu, "Minimizing electricity cost: Optimization of distributed internet data centers in a multi-electricity-market environment," in *2010 Proceedings IEEE INFOCOM*. IEEE, 2010, pp. 1–9.
- [6] M. Lin, A. Wierman, L. L. Andrew, and E. Thereska, "Dynamic right-sizing for power-proportional data centers," *IEEE/ACM Transactions on Networking*, vol. 21, no. 5, pp. 1378–1391, 2012.
- [7] N. H. Tran, D. H. Tran, S. Ren, Z. Han, E.-N. Huh, and C. S. Hong, "How geo-distributed data centers do demand response: A game-theoretic approach," *IEEE Transactions on Smart Grid*, vol. 7, no. 2, pp. 937–947, 2015.
- [8] H. Wang, J. Huang, X. Lin, and H. Mohsenian-Rad, "Proactive demand response for data centers: A win-win solution," *IEEE Transactions on Smart Grid*, vol. 7, no. 3, pp. 1584–1596, 2015.
- [9] Í. Goiri, W. Katsak, K. Le, T. D. Nguyen, and R. Bianchini, "Parasol and greenswitch: Managing datacenters powered by renewable energy," *ACM SIGPLAN Notices*, vol. 48, no. 4, pp. 51–64, 2013.
- [10] G. Buttazzo, G. Lipari, L. Abeni, and M. Caccamo, *Soft real-time systems: predictability vs. efficiency*. Springer, 2005.
- [11] Z. Xiao, W. Song, and Q. Chen, "Dynamic resource allocation using virtual machines for cloud computing environment," *IEEE transactions on parallel and distributed systems*, vol. 24, no. 6, pp. 1107–1117, 2012.
- [12] Z. Zhou, F. Liu, Y. Xu, R. Zou, H. Xu, J. C. Lui, and H. Jin, "Carbon-aware load balancing for geo-distributed cloud services," in *2013 IEEE 21st International Symposium on Modelling, Analysis and Simulation of Computer and Telecommunication Systems*. IEEE, 2013, pp. 232–241.
- [13] M. Padberg and G. Rinaldi, "A branch-and-cut algorithm for the resolution of large-scale symmetric traveling salesman problems," *SIAM review*, vol. 33, no. 1, pp. 60–100, 1991.
- [14] L. A. Wolsey, *Integer programming*. John Wiley & Sons, 2020.